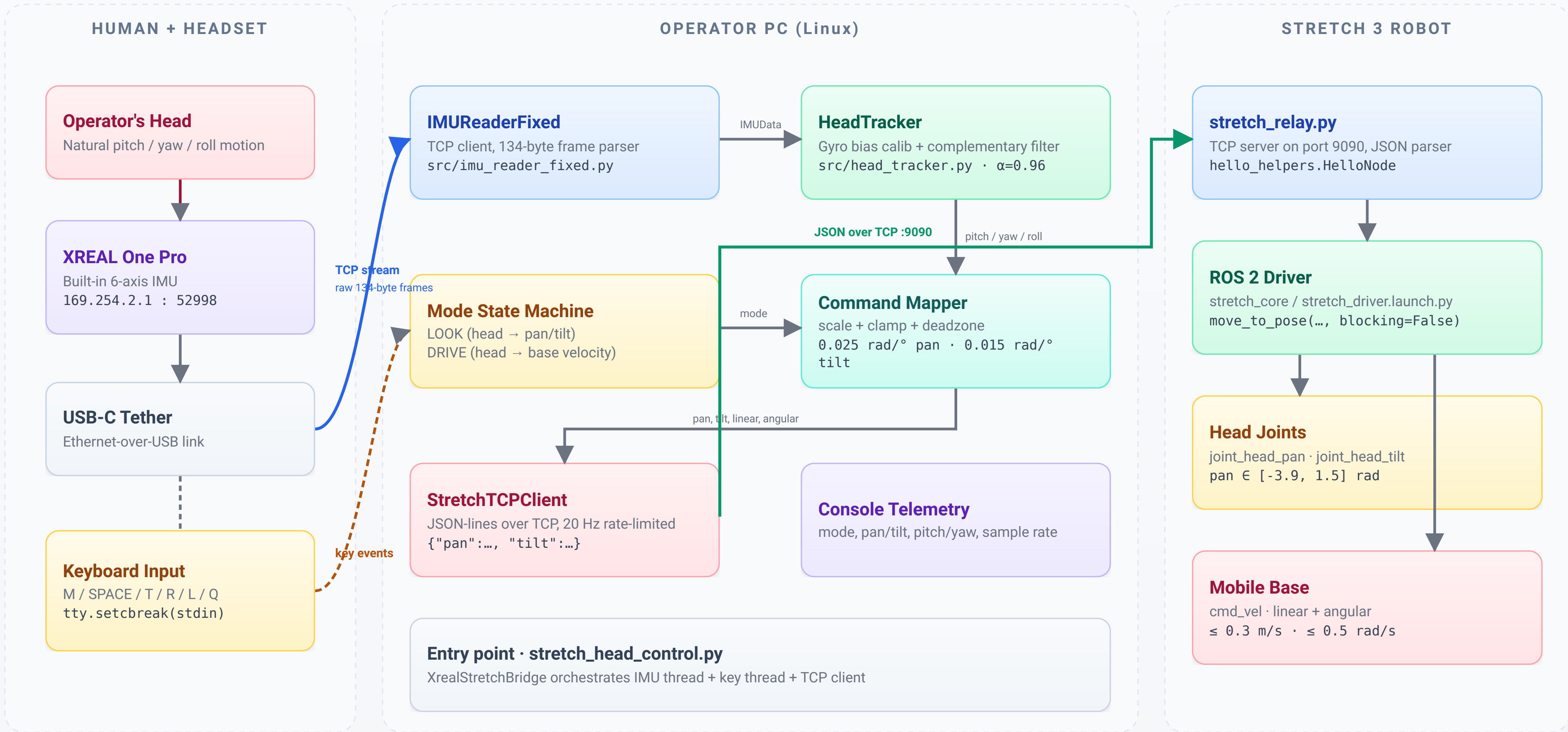
XREAL-Controlled Stretch 3 — System Architecture

End-to-end data path: head motion → PC bridge → robot actuation

PAGE 1 · OVERVIEW

GAZEBOT · TELEOP STACK



Breaking Into the XREAL One Pro — TCP Relay Driver

From opaque USB-C stream to parsed IMU frames on Linux

1 · LINK-LAYER DISCOVERY

Plug XREAL via USB-C
Enable "ethernet" in glasses developer menu
`ping 169.254.2.1`

Glasses expose TCP service
Unofficial sensor stream, unauthenticated
`IP 169.254.2.1 · PORT 52998`

socket.connect()
Single blocking TCP client, 5 s timeout
`AF_INET · SOCK_STREAM`

Observation
Stream is chunked by TCP (4096-byte recvs)—no length prefix, no JSON. A repeating 6-byte magic appears at fixed cadence.
`HEADER = 28 36 00 00 00 80`

raw bytes

2 · MESSAGE FRAMING

recv_buffer += sock.recv(4096)
Append to rolling buffer until two HEADERs are present

Header-to-header framing
Find HEADER at h1, find next HEADER at h2 ⇒ one complete message = buffer[h1:h2]
`msg length ≈ 134 bytes`

Secondary sentinel
Locate SENSOR marker — end of the IMU block
`SENSOR_MSG = 00 40 1F 00 00 40`

Message layout (reverse-engineered)
[0–5] header · **[6–13]** session · **[14–15]** invariant
[16–25] static · **[26–33]** flags/counter
[34–57] 6× float32 (LE) · then SENSOR_MSG

framed msg

3 · IMU EXTRACTION

Slice IMU section
msg[26 : sensor_pos] → skip 8 flag bytes
`data_offset = 8 if len≥32`

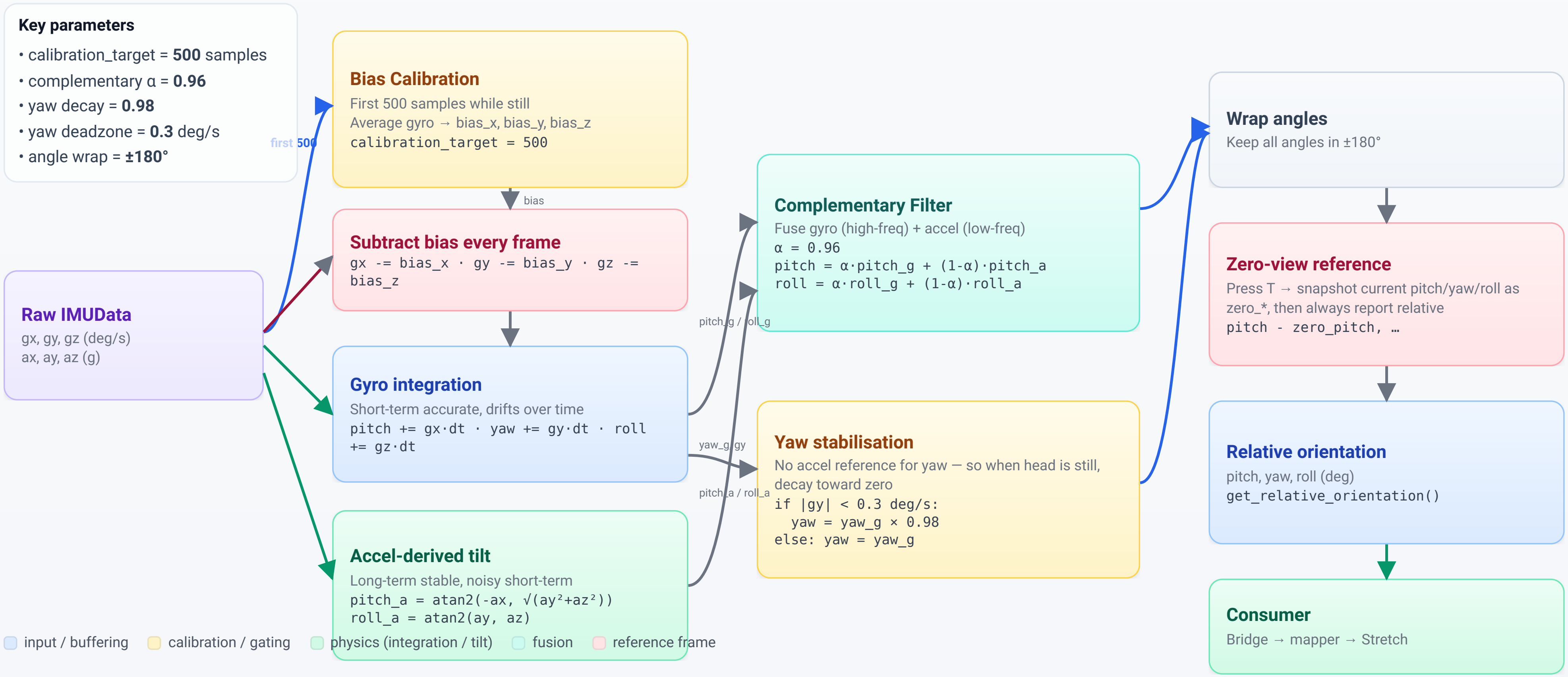
Unpack 6 little-endian floats
3 gyroscope (rad/s) + 3 accelerometer (g)
`struct.unpack('<f', …) × 6`

Convert & remap axes
gyro → deg/s (× 180/π), accel axes swap
`ax=v[5], ay=v[4], az=v[3]`

Emit IMUData → callback
Feeds HeadTracker and the bridge at native device rate (~100 Hz typical)
`callback(imu_data, count, rate)`

IMU Configuration & Sensor Fusion Pipeline

From raw gyro/accel samples to stable head orientation



Operator Controls — Modes, Key Bindings & Robot Outputs

How keystrokes and head motion combine into LOOK / DRIVE commands

Keyboard Bindings

Captured via tty.setcbreak on stdin; non-blocking, case-insensitive

Key	Action	Notes
`M`	**Toggle LOOK / DRIVE**	Emits cmd_vel = 0 on switch to LOOK
`SPACE`	**Hold to drive**	Gate for DRIVE mode; 0.2 s release timeout
`T`	**Zero view**	Sets current head pose as neutral
`R`	**Recalibrate**	Recollect 500 stillness samples
`L`	**Lock tracking**	Freeze outputs; stops base if DRIVE
`Q`	**Quit**	Sends stop, os._exit(0)

Rate limit: 20 Hz per command (COMMAND_INTERVAL = 0.05 s)

State: LOOK

Head pose → robot head pan/tilt
Default on boot

State: DRIVE

Head pose + SPACE → base velocity

M

after 100%

pan / tilt

L

State: LOCKED

Telemetry only · no commands sent

State: CALIBRATING

500 stillness samples required

LOOK mapping → head joints

Pan and tilt are proportional to head yaw and pitch

```
pan_rad  = yaw × 0.025   (HEAD_PAN_SCALE)
tilt_rad = -pitch × 0.015   (HEAD_TILT_SCALE)
clamp pan ∈ [-3.9, 1.5] rad · tilt ∈ [-1.53, 0.79] rad
rate 20 Hz · JSON {"pan":..,"tilt":..} → :9090
```

DRIVE mapping → base velocity

Only while SPACE has been pressed within the last 0.2 s

```
deadzone = 5° · max angle = 30°
scaled = (|θ| - 5) / (30 - 5)  → clamp [0, 1]

linear  = sign(-pitch) · scaled · 0.3 m/s
angular = sign(-yaw)   · scaled · 0.5 rad/s

SPACE released → send (0, 0) immediately
```

cmd_vel

Robot: head joints

Via stretch_relay.py · move_to_pose non-blocking

```
joint_head_pan
joint_head_tilt
```

Robot: mobile base

cmd_vel (Twist) over ROS 2

```
linear.x · angular.z
```

Source: stretch_head_control.py · _key_loop() · imu_callback() · StretchTCPClient